

PLAN DE SÉANCE

Programmation Web 1

Cours Programmation Web 1	Thème de la séance Interaction avec JavaScript
Niveau Débutants complets	Durée 3 heures (180 minutes)
Prérequis Notions HTML et CSS de base	Outils VS Code, navigateur Chrome/Firefox

OBJECTIFS DE LA SÉANCE

À la fin de cette séance, l'apprenant sera capable de :

O1	Comprendre le rôle de JavaScript dans une page web et sa relation avec le DOM
O2	Sélectionner et manipuler des éléments HTML dynamiquement avec JavaScript
O3	Attacher des événements (click, input, submit) à des éléments de la page
O4	Créer et valider un formulaire interactif en JavaScript pur
O5	Appliquer des effets visuels simples (animations CSS déclenchées par JS)

DÉROULEMENT DE LA SÉANCE

Durée	Phase	Activité	Ressources
10 min	Mise en route	Accueil, présentation du plan, vérification des prérequis (quiz rapide oral)	Tableau blanc
20 min	Cours Magistral	Introduction à JS : qu'est-ce que le DOM ? Comment JS interagit avec HTML ?	Diaporama
30 min	Démonstration	Manipulation du DOM : getElementById, querySelector, innerHTML, classList	Éditeur live
10 min	Pause	Pause active — les apprenants testent un snippet fourni dans leur éditeur	VS Code
35 min	Atelier 1	Événements JS : onclick, addEventListener, gestion du click et de l'input	Exercices guidés

Durée	Phase	Activité	Ressources
35 min	Atelier 2	Formulaire avec validation JS et retour visuel (messages d'erreur dynamiques)	Projet starter
20 min	Atelier 3	Animations CSS déclenchées par JS : toggle de classes, transitions	Projet starter
20 min	Synthèse	Mini-projet intégratif : carte d'identité interactive (DOM + events + form + anim)	Projet final
00 min	Clôture	Bilan oral, Q&R, présentation du travail à remettre	Tableau blanc

SECTION 1 — MANIPULATION DU DOM



Concept

Le DOM (Document Object Model) est la représentation en mémoire de votre page HTML. JavaScript peut lire et modifier chaque élément, attribut ou texte de cette structure en temps réel.

1.1 — C'est quoi le DOM ?

Quand un navigateur charge une page HTML, il construit une arborescence d'objets appelée DOM. Chaque balise HTML devient un nœud dans cet arbre. JavaScript peut accéder à n'importe quel nœud, le modifier, en ajouter ou en supprimer.

1.2 — Sélectionner des éléments

Il existe plusieurs méthodes pour cibler un élément dans le DOM :

```
// Par identifiant unique
const titre = document.getElementById('monTitre');

// Par sélecteur CSS (retourne le PREMIER résultat)
const bouton = document.querySelector('.btn-primary');

// Par sélecteur CSS (retourne TOUS les résultats)
const items = document.querySelectorAll('li.item');

// Par nom de balise
const paragraphes = document.getElementsByTagName('p');
```

1.3 — Modifier le contenu et le style

```
const el = document.getElementById('resultat');

// Modifier le texte (sans HTML)
el.textContent = 'Bonjour tout le monde !';
```

```
// Injecter du HTML
el.innerHTML = '<strong>Bravo !</strong> Tu as réussi.';

// Modifier le style directement
el.style.color = 'blue';
el.style.fontSize = '20px';

// Ajouter / retirer une classe CSS
el.classList.add('actif');
el.classList.remove('cache');
el.classList.toggle('surligne'); // ajoute OU retire selon l'état
```

1.4 — Créer et insérer des éléments

```
// Créer un nouveau paragraphe
const p = document.createElement('p');
p.textContent = 'Nouvel élément créé avec JS';
p.classList.add('note');

// L'insérer dans le body
document.body.appendChild(p);

// L'insérer AVANT un autre élément
const ref = document.getElementById('section2');
document.body.insertBefore(p, ref);
```

Exercice 1

Ouvrir la console du navigateur (F12). Sélectionner le titre de la page, changer son texte, sa couleur, et lui ajouter une classe CSS 'surligne'. Durée : 10 minutes.

SECTION 2 — ÉVÉNEMENTS JAVASCRIPT

Concept

Un événement est une action déclenchée par l'utilisateur (clic, frappe, survol...) ou par le navigateur (chargement, erreur...). JavaScript permet d'écouter ces événements et d'y répondre avec une fonction.

2.1 — Méthodes d'attachement d'événements

```
// ❌ Méthode ancienne (à éviter — écrase l'existant)
bouton.onclick = function() { console.log('cliqué'); };

// ✅ Méthode moderne recommandée
bouton.addEventListener('click', function() {
  console.log('Bouton cliqué !');
});
```

```
});

// Avec une fonction nommée (meilleure lisibilité)
function handleClick() {
  console.log('Bouton cliqué !');
}
bouton.addEventListener('click', handleClick);
```

2.2 — Types d'événements courants

Événement	Déclenché quand...	Exemple d'usage
click	L'utilisateur clique	Soumettre un formulaire
input	La valeur d'un champ change	Validation en temps réel
change	Sélection modifiée (select, checkbox)	Filtrer une liste
submit	Formulaire soumis	Valider avant envoi
mouseover	Survol d'un élément	Afficher un tooltip
keydown	Touche du clavier enfoncée	Raccourcis clavier
DOMContentLoaded	Page entièrement chargée	Initialiser l'app

2.3 — L'objet événement (event)

```
document.addEventListener('click', function(event) {
  // event contient des informations sur ce qui s'est passé
  console.log('Élément cliqué :', event.target);
  console.log('Position X :', event.clientX);
  console.log('Position Y :', event.clientY);
});

// Bloquer le comportement par défaut (ex: envoi de formulaire)
formulaire.addEventListener('submit', function(event) {
  event.preventDefault(); // Empêche le rechargement de la page
  // Traitement personnalisé ici
});
```

2.4 — Exemple complet : Compteur interactif

```
<!-- HTML -->
<div id='compteur'>0</div>
<button id='btnPlus'>+</button>
<button id='btnMoins'>-</button>
<button id='btnReset'>Reset</button>

// JavaScript
let valeur = 0;
const affichage = document.getElementById('compteur');
```

```
document.getElementById('btnPlus').addEventListener('click', () => {
  valeur++;
  affichage.textContent = valeur;
});

document.getElementById('btnMoins').addEventListener('click', () => {
  valeur--;
  affichage.textContent = valeur;
});

document.getElementById('btnReset').addEventListener('click', () => {
  valeur = 0;
  affichage.textContent = valeur;
});
```



Exercice 2

Créer une page avec un champ de texte et un paragraphe. À chaque frappe dans le champ, afficher en temps réel dans le paragraphe ce que l'utilisateur écrit (événement 'input'). Ajouter un bouton 'Effacer'. Durée : 15 minutes.

SECTION 3 — FORMULAIRES & VALIDATION JS



Concept

La validation JS côté client améliore l'expérience utilisateur en signalant les erreurs avant tout envoi au serveur. Elle NE remplace pas la validation serveur — elle la complète.

3.1 — Lire la valeur des champs

```
// Champ texte
const nom = document.getElementById('nom').value;

// Case à cocher
const accepte = document.getElementById('cgu').checked;

// Liste déroulante
const pays = document.getElementById('pays').value;

// Groupe de boutons radio
const genre = document.querySelector('input[name="genre"]:checked').value;
```

3.2 — Structure d'une validation complète

```
const formulaire = document.getElementById('monFormulaire');

formulaire.addEventListener('submit', function(event) {
```

```
event.preventDefault(); // Bloquer l'envoi par défaut

let valide = true;

// --- Validation du nom ---
const nom = document.getElementById('nom').value.trim();
if (nom === '') {
  afficherErreur('nom', 'Le nom est obligatoire.');
```

valide = false;

```
} else {
  effacerErreur('nom');
}

// --- Validation de l'email ---
const email = document.getElementById('email').value.trim();
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  afficherErreur('email', 'Adresse email invalide.');
```

valide = false;

```
} else {
  effacerErreur('email');
}

// Si tout est valide, envoyer
if (valide) {
  console.log('Formulaire valide. Envoi...');
```

}

```
});

// Fonctions utilitaires
function afficherErreur(champ, message) {
  const erreur = document.getElementById('erreur-' + champ);
  if (erreur) {
    erreur.textContent = message;
    erreur.style.display = 'block';
  }
}

function effacerErreur(champ) {
  const erreur = document.getElementById('erreur-' + champ);
  if (erreur) erreur.style.display = 'none';
}
```

3.3 — HTML du formulaire associé

```
<form id='monFormulaire' novalidate>
  <div>
    <label for='nom'>Nom :</label>
    <input type='text' id='nom' placeholder='Votre nom'>
    <span id='erreur-nom' class='erreur' style='display:none; color:red'></span>
  </div>
  <div>
    <label for='email'>Email :</label>
    <input type='email' id='email' placeholder='vous@email.com'>
    <span id='erreur-email' class='erreur' style='display:none; color:red'></span>
```

```
</div>
<button type='submit'>Envoyer</button>
</form>
```

Exercice 3

Créer un formulaire d'inscription avec : prénom (obligatoire, min 2 caractères), email (format valide), mot de passe (min 8 caractères), confirmation de mot de passe (identique au précédent). Afficher les erreurs sous chaque champ en rouge. Afficher un message de succès vert si tout est correct. Durée : 20 minutes.

SECTION 4 — ANIMATIONS & EFFETS VISUELS

Concept

La bonne pratique est de définir les animations en CSS et de les déclencher en JavaScript via l'ajout ou le retrait de classes. Cela sépare le style (CSS) de la logique (JS).

4.1 — Principe : Toggle de classes CSS

```
/* CSS — Définir les états et transitions */
.boite {
  width: 100px;
  height: 100px;
  background: #2E6DB4;
  transition: all 0.4s ease; /* Animation douce */
}

.boite.agrandi {
  width: 200px;
  height: 200px;
  background: #E8A020;
  border-radius: 50%;
}

// JavaScript — Déclencher l'animation
const boite = document.querySelector('.boite');

boite.addEventListener('click', function() {
  boite.classList.toggle('agrandi');
});
```

4.2 — Afficher / Masquer avec animation

```
/* CSS */
.panneau {
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.5s ease;
```

```
}  
.panneau.ouvert {  
  max-height: 300px;  
}  
  
// JavaScript  
const btnToggle = document.getElementById('toggleBtn');  
const panneau = document.getElementById('panneau');  
  
btnToggle.addEventListener('click', function() {  
  panneau.classList.toggle('ouvert');  
  btnToggle.textContent = panneau.classList.contains('ouvert')  
    ? 'Masquer' : 'Afficher';  
});
```

4.3 — Animation au scroll

```
// Révéler un élément quand il entre dans la fenêtre  
const sections = document.querySelectorAll('.section-animee');  
  
window.addEventListener('scroll', function() {  
  sections.forEach(section => {  
    const rect = section.getBoundingClientRect();  
    const visible = rect.top < window.innerHeight - 100;  
    if (visible) {  
      section.classList.add('visible');  
    }  
  });  
});  
  
/* CSS */  
.section-animee {  
  opacity: 0;  
  transform: translateY(40px);  
  transition: opacity 0.6s ease, transform 0.6s ease;  
}  
.section-animee.visible {  
  opacity: 1;  
  transform: translateY(0);  
}
```

Exercice 4

Créer une page de présentation d'un produit fictif avec : (1) un bouton qui agrandit/réduit une image au clic, (2) un accordéon FAQ (questions/réponses qui se déploient), (3) au moins 3 sections qui apparaissent au scroll. Durée : 20 minutes.

MINI-PROJET INTÉGRATIF FINAL



Réaliser une Carte d'Identité Interactive qui mobilise les 4 compétences de la séance : DOM, Événements, Formulaire + Validation, Animations.

Énoncé

Créez une page web avec deux zones côte à côte :

- À gauche : un formulaire avec les champs Prénom, Nom, Profession, Email et une couleur de thème (color picker)
- À droite : une carte de prévisualisation qui se met à jour EN TEMPS RÉEL à chaque saisie

Fonctionnalités attendues :

- Prévisualisation instantanée du nom, prénom et profession dans la carte
- La couleur de thème change le fond de la carte avec une transition CSS
- Un bouton 'Valider & Exporter' qui valide tous les champs et affiche un message de succès animé
- Un bouton 'Réinitialiser' qui efface tout avec une animation de disparition